The Stack That Fits Any Project + Projects Worth Building

Learn this stack once, reuse it in every project. The tools matter less than the fact that your project **has these layers**.

Stack

1. **API** - FastAPI / Spring Boot / NestJS. Clean routes, validation, error handling.
2. **Database** - PostgreSQL by default. Real schema, indexes, one query you optimised.
3. **Cache** - Redis. Caching, rate limiting, sessions. Know your hit ratio.
4. **Queue / Events** - Kafka or RabbitMQ. Anything slow goes async: emails, scraping, AI calls, video jobs.
5. **AI layer** - an LLM API (OpenAI / Anthropic / Gemini) or an open-weight model self-hosted with Ollama - Qwen, Mistral, Gemma. Add pgvector only if you do retrieval.
6. **Containers** - Docker + docker-compose for local dev.
7. **CI/CD** - GitHub Actions: push → tests run → Docker image builds → auto-deploys. Even a 20-line pipeline puts you ahead of 95% of student repos.
8. **Kubernetes** - run services on a local cluster (kind or k3s), then show autoscaling: load test hits, HPA spins up pods, Grafana shows it. That 30-second demo **is** the interview.

Observability

- **Traces:** OpenTelemetry → view in Jaeger/Zipkin. One trace should visibly fan out: *request → Redis hit/miss → Kafka → worker → LLM call (ms) → DB write*.
- **Metrics:** Prometheus → Grafana. Show req/s, p95 latency, error rate, queue depth.
- **Logs:** Kibana (ELK) or Grafana Loki, with request IDs.

All three run locally as ~30 lines in docker-compose. Put a GIF of the trace + dashboard in your README.

6 Projects ideas

1. Remote Code Execution Engine *(mini LeetCode judge)*
Users submit code → queue → sandboxed Docker workers run it → results stream back. Autoscale workers on k8s under load.
Flex: sandboxing, timeouts, "ran 500 concurrent submissions."
2. Mini YouTube *(video pipeline)*
Upload → Kafka → ffmpeg workers transcode to multiple qualities → HLS streaming with Redis caching.
Flex: "processed a 1GB upload without blocking a single request."
3. Trading Matching Engine *(mini Zerodha)*
In-memory order book matching buy/sell orders, Kafka event log, live order-book UI over WebSockets, replay-from-events on crash.
Flex: "matched 10k orders/sec, exactly-once, crash-recoverable."

4. Mini Datadog *(log intelligence platform)*
Other apps ship logs/metrics to YOUR system → anomaly detection flags spikes → LLM writes the incident summary with probable root cause.
Flex: you didn't just use observability - you built one.
5. Collaborative Code Editor *(mini Google Docs for code)*
Multiple live cursors via CRDTs (use Yjs), presence via Redis pub/sub, snapshots to Postgres.
Flex: explaining conflict resolution beats explaining CRUD forever.
6. Search Engine for Your College *(mini Google)*
Distributed crawler (queued, polite, deduped) over college sites + PYQs → hybrid search (Elasticsearch + embeddings) → LLM answers with citations.
Flex: real users guaranteed.

⚠️ **Two rules:** build **one** to full depth, not all six. And every layer must earn its place, and you must be able to say why in one line.

Show It or It Didn't Happen

- Live link at the top of the README. Trace + Grafana GIF right below it.
- One resume bullet with a number: *"cut p95 from 800ms to 90ms with Redis"* / *"HPA scaled 2→10 pods under k6 load."*
- Pin your best 2–3 repos, archive the dead ones.
- Projects get you the interview; **DSA converts it into the offer.** Do both.

Links

- CI/CD: GitHub Actions · Kubernetes: kind, k3s
- Observability: OpenTelemetry, Grafana, Jaeger
- Core: Kafka, Redis, Yjs, k6 load testing
- Deploy free: Render, Railway, Fly.io
- DSA: NeetCode, LeetCode